A
Project Report On

Gujarati-Speech-To-Text

By:

Preya Dhangar(CE083)(23CEUTS036)

Rudra Kalariya(CE104)(23CEUOS065)

Kalpesh Gangani(CE028)(23CEUOG047)

B. Tech CE, Semester VI

Subject: System Design Practices

Guided By:

Prof. Brijesh S. Bhatt

Assistant Professor

Dept. of Computer Engineering,

Department of Computer Engineering

Faculty of Technology

Dharmsinh Desai University

Faculty of Technology
Department of Computer Engineering
Dharmsinh Desai University

CERTIFICATE

This is to certify that the practical/term work carried out in the subject of System Design Practices and recorded in this journal is the bonafide work of

Preya Dhangar, ID NO:23CEUTS036
Rudra Kalariya, ID NO:23CEUOS065
Kalpesh Gangani,ID NO:23CEUOG047

of B.Tech semester VI in the branch of Computer Engineering during the academic year 2025-2026

Prof.Brijesh S Bhatt
Assistant Professor,
Dept. of Computer Engineering,
Faculty of Technology,
Dharmsinh Desai University,
Nadiad

Dr. C. K. Bhensdadia
Head,
Dept. of Computer Engineering,
Faculty of Technology,
Dharmsinh Desai University,
Nadiad

Acknowledgement

It is indeed a great pleasure to express my thanks and gratitude to all those who helped me during this project. This project would have been materialized without the help from many who asked me good questions and rescued me from various red tape crises.

Theoretical Knowledge is of no importance if one doesn't know the way of its implementation. I am thankful to our institute that provided me an opportunity to apply my theoretical knowledge through the project. I feel obliged to submit this project as part of our curriculum.

I would like to take the opportunity to express my humble gratitude to my guide Prof. Brijesh S Bhatt, under whom I undertook my project work. His constant guidance and willingness to share his vast knowledge made me enhance my knowledge and helped me to complete the assigned tasks to perfection. Without his effort, support and an astonishing testing ability this project may not have succeded.

With Sincere Regards,

Preya ,Rudra, Kalpesh

Table of Contents

CERTIFICATE

Acknowledgement Abstract

1.Introduction.....5

2.SoftwareRequirementsSpecification..... 8

3.Design.....18

3.1UseCaseDiagram.....18

3.2ClassDiagram.....19

3.3SequenceDiagram.....21

3.4ActivityDiagram.....22

3.5FlowChart.....25

4.Implementation.....26

5.Testing.....29

6.Screenshots.....32

7.Conclusion.....34

1. Introduction

The Gujarati Custom ASR Pipeline project aims to provide an efficient and accurate platform for transcribing Gujarati speech into text using advanced Artificial Intelligence techniques. Automatic Speech Recognition (ASR) plays a crucial role in enabling voice-based applications, improving accessibility, and supporting regional languages in the digital ecosystem.

Gujarati speech transcription presents unique challenges such as complex phonetic structures, inconsistent Unicode representation, and handling of matras (dependent vowel signs). This project addresses these challenges by developing a custom two-stage ASR pipeline that ensures both high transcription accuracy and proper linguistic structure.

The system integrates a fine-tuned Whisper-based acoustic model with linguistic post-processing mechanisms to deliver clean, grammatically correct, and standardized Gujarati text output.

1.1 Technology Used

The Gujarati Custom ASR Pipeline is developed using modern technologies and frameworks to ensure high performance, scalability, and accuracy in speech recognition.

- **Programming Language:**
  Python – used for implementing the entire pipeline, including model integration and backend logic.
- **Machine Learning Frameworks:**
  Hugging Face Transformers – for building and deploying the ASR model
  PyTorch – for model training and fine-tuning
- **Speech Recognition Model:**
  Whisper-Small (Fine-Tuned) – used as the core acoustic model for Gujarati speech recognition

- **Natural Language Processing Tools:**
  IndicNLP Library / Smruti API – for text normalization, grammar correction, and Unicode handling
- **Web Framework / UI:**
  Gradio – for creating an interactive and user-friendly web interface
- **Dataset Used:**
  Kathbath Dataset – for domain-specific Gujarati speech training
  FLEURS Dataset – for evaluation and benchmarking
- **Audio Processing:**
  Librosa / FFmpeg – for audio preprocessing, resampling, and format handling
- **Version Control & Hosting:**
  Git & GitHub – for code management
  Hugging Face Hub – for hosting trained model weights

1.2 Platform Used

The Gujarati Custom ASR Pipeline is developed and deployed across multiple platforms to ensure smooth development, testing, and user interaction.

- **Development Platform:**
  The system is developed using Python in environments such as Jupyter Notebook and Visual Studio Code (VS Code) for model training, testing, and debugging.
- **Operating System:**
  The project is compatible with major operating systems including Windows, Linux, and macOS, ensuring flexibility for developers and users.

- **Model Hosting Platform:**
  Hugging Face Hub is used to host the fine-tuned ASR model, allowing easy access, downloading, and integration during deployment.
- **Web Deployment Platform:**
  The application is deployed locally using Gradio, which provides a browser-based interface accessible via localhost.
- **Version Control Platform:**
  GitHub is used for managing source code, collaboration, and maintaining project versions.
- **Execution Environment:**
  The system runs in a local machine environment with required dependencies installed, and supports GPU acceleration (if available) for faster processing.

2. Software Requirements Specification (SRS)

2.1 Introduction

1. Purpose

The purpose of this project is to design and develop a user-friendly Automatic Speech Recognition (ASR) platform specifically for the Gujarati language. The system aims to accurately convert spoken Gujarati audio into structured and grammatically correct text using advanced machine learning and natural language processing techniques.

This project focuses on addressing challenges such as phonetic complexity, noisy audio inputs, and Unicode inconsistencies. It provides an efficient solution for real-world applications like transcription, accessibility tools, and voice-enabled systems, thereby promoting the use of regional languages in digital platforms.

2. Product Scope

This project involves the development of a **web-based Gujarati Speech-to-Text platform** that enables users to upload or record audio and receive accurate text transcription.

The system includes:

- A **two-stage ASR pipeline** (Acoustic Model + Linguistic Processing)
- **Audio preprocessing features** such as resampling and chunking
- **Text normalization and grammar correction** for clean output
- A **user-friendly web interface** for easy interaction
- Integration with **pre-trained and fine-tuned models** hosted online

The platform ensures high accuracy, efficient processing, and scalability, making it suitable for both individual and organizational use.

3. References

- *Fundamentals of Software Engineering* by Rajib Mall, PHI Learning
- IEEE Std 830-1998 – *IEEE Recommended Practice for Software Requirements Specifications*
- Hugging Face Documentation (Transformers & Model Deployment)
- OpenAI Whisper Model Documentation
- IndicNLP Library Documentation

2.2 Overall Description

1. Product Perspective

The Gujarati Custom ASR Pipeline operates as a **web-based speech recognition platform** designed specifically for the Gujarati language. It serves as a part of the broader ecosystem of Artificial Intelligence and Natural Language Processing applications, enabling voice-based interaction and transcription services.

The system complements existing speech recognition technologies by focusing on **regional language support**, improving accessibility, and addressing challenges unique to Gujarati speech and text processing.

2. Product Functions

ADMIN

- Monitor system performance and usage
- Manage uploaded audio records (if stored)
- Oversee model updates and system maintenance

USER

- Upload or record Gujarati audio
- Convert speech into text (Speech-to-Text)
- View and download transcribed output
- Re-upload or retry audio for improved results
- Use the system through an interactive web interface

3. User Classes and Characteristics

- **Regular Users:**
  Individuals such as students, professionals, or content creators who want to convert Gujarati speech into text. They require a simple and intuitive interface with minimal technical knowledge.
- **Developers / Researchers:**
  Users interested in experimenting with ASR models, analyzing results, or integrating the system into other applications.
- **Administrators:**
  Responsible for managing users, maintaining the system, monitoring performance, and ensuring smooth functioning of the platform.

4. Operating Environment

The system operates in a **web-based environment** and is accessible through modern web browsers.

- Backend developed using **Python-based frameworks**
- Model inference using **Hugging Face Transformers and PyTorch**
- Frontend powered by **Gradio interface**
- Compatible with **Windows, Linux, and macOS**
- Requires a **stable internet connection** (for model loading and processing)

5. User Documentation

- **User Manual:**
  Provides step-by-step instructions on how to upload audio, record voice, and obtain transcription results.
- **Video Tutorials:**
  Demonstrates how to use the platform, including recording audio, uploading files, and interpreting outputs.

6. Assumptions and Dependencies

Assumptions

- Users have basic knowledge of using web applications
- Audio input provided is in Gujarati language
- The system performance may vary based on audio quality and background noise
- Internet connectivity is available for model loading and execution

Dependencies

- **Hugging Face Transformers:** For loading and running ASR models
- **PyTorch:** For model execution and inference
- **Gradio:** For building the user interface
- **FFmpeg / Librosa:** For audio preprocessing
- **IndicNLP / Smruti API:** For text normalization and grammar correction
- **Model Hosting (Hugging Face Hub):** For accessing trained model weights

2.3 External Interfaces

1. Hardware Interface

- **User Device (Desktop/Laptop):**
  The system is accessed through a web browser and requires basic hardware such as CPU, memory, microphone (for recording), and display screen.
- **Server / Processing System:**
  The ASR model runs on a local or cloud-based system that requires sufficient CPU/GPU, memory, and storage for processing audio and generating transcriptions.

2. Communication Interface

- The frontend interacts with the backend through **internal function calls and API-like processing pipelines**
- Audio data is passed to the model for inference, and the processed text is returned to the interface
- Standard data exchange formats are used for handling audio and text

2.4 System Features

Functional Requirements

R.1 User Interaction (Accessing the System)

Description:
This functionality allows users to access the ASR platform and interact with the system through a simple web interface without complex authentication requirements.

R.1.1 Access Platform

- Input: User opens the web application in a browser
- Output: User is directed to the ASR interface
- Processing: System loads the interface and initializes required components

R.2 Audio Input Processing

Description:
This functionality allows users to provide Gujarati audio input either by uploading a file or recording directly through the system.

R.2.1 Upload Audio File

- Input: User uploads an audio file (e.g., WAV, MP3)
- Output: Audio is accepted and prepared for processing
- Processing: System validates format, resamples audio to 16kHz, and prepares chunks

R.2.2 Record Audio

- Input: User records audio using a microphone
- Output: Recorded audio is captured and sent for processing
- Processing: System captures audio input and converts it into a compatible format

R.3 Speech-to-Text Conversion

Description:
This core functionality converts Gujarati speech into text using the ASR pipeline.

R.3.1 Transcribe Audio

- Input: Preprocessed Gujarati audio
- Output: Raw transcribed Gujarati text
- Processing: Whisper-based model processes audio and generates transcription

R.4 Text Post-Processing

Description:
This functionality improves the quality of transcribed text by correcting grammar and formatting issues.

R.4.1 Normalize Text

- Input: Raw transcribed text
- Output: Clean, grammatically correct Gujarati text
- Processing: IndicNLP / Smruti API fixes Unicode errors, matras, and sentence structure

R.5 Output Display and Download

Description:

This functionality allows users to view and store the transcription results.

R.5.1 Display Output

- Input: Processed text
- Output: Text displayed on the user interface
- Processing: System renders final transcription on the web page

R.5.2 Download Output

- Input: User clicks download option
- Output: Text file is downloaded
- Processing: System generates and provides downloadable text file

R.6 Cross-Platform Compatibility

Description:

Ensures the system works seamlessly across devices.

R.6.1 Android Compatibility

- Input: User accesses system via Android browser
- Output: Fully functional interface
- Processing: Responsive UI adapts to mobile screens

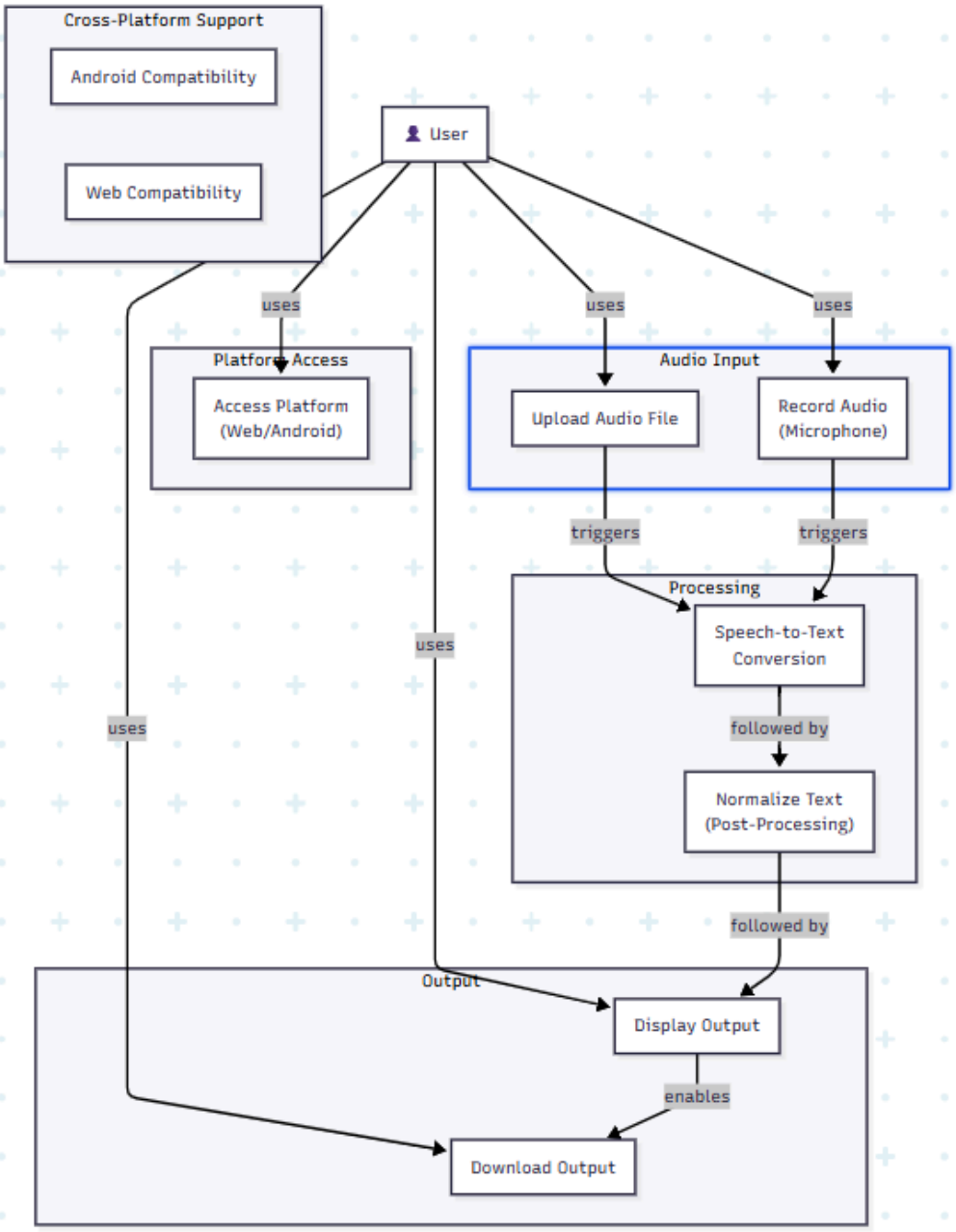
R.6.2 Web Compatibility

- Input: User accesses via desktop browser

- Output: Full-featured interface
- Processing: System renders optimized layout

3. Design

3.1 Use Case Diagram



3.2 Class Diagram

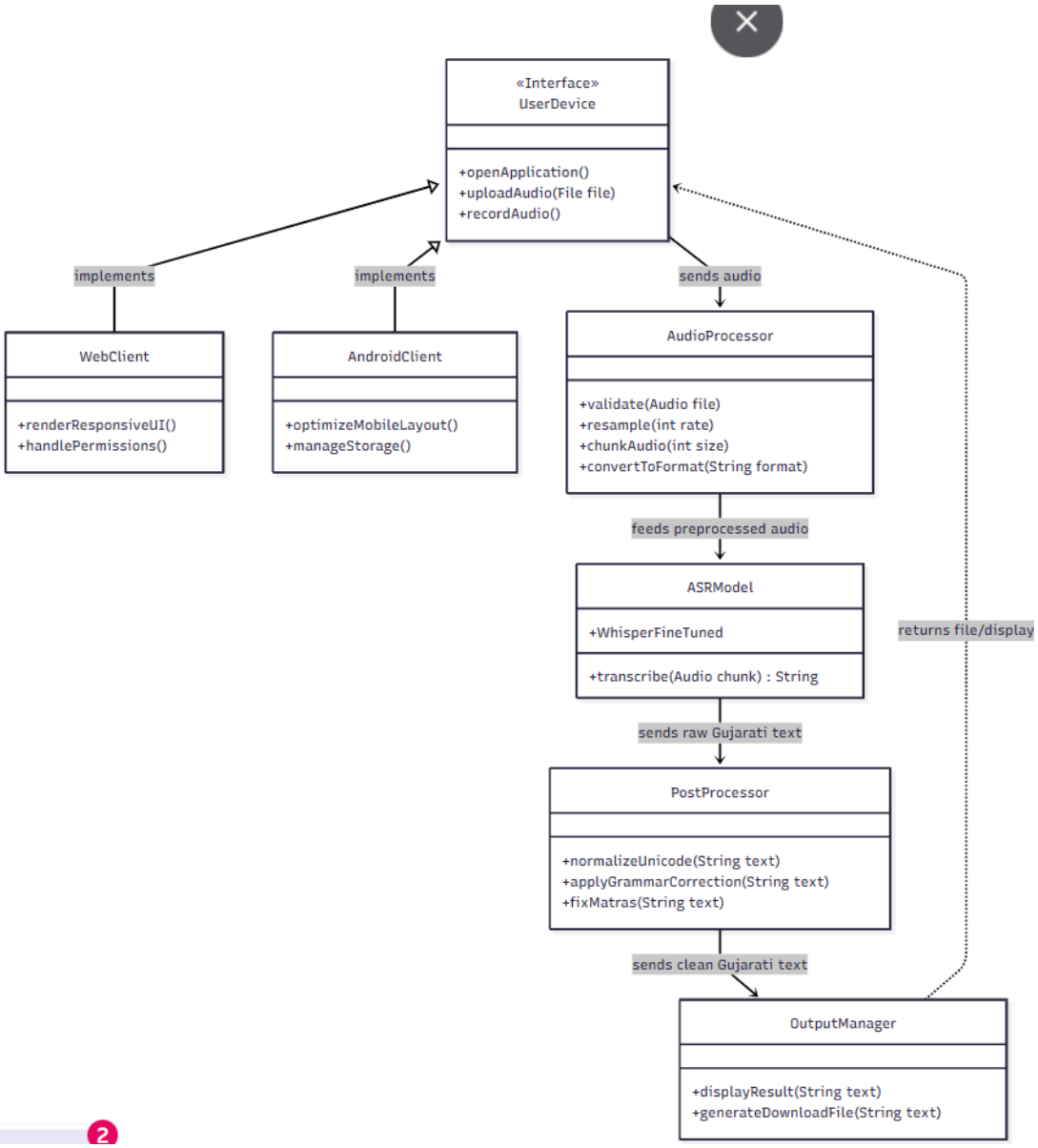


Diagram Description (ASR Model Flow)

The diagram represents the **working of the ASR (Automatic Speech Recognition) model** within your Gujarati ASR pipeline.

1. **Input: Preprocessed Audio**
   - The process starts with **preprocessed audio input**.
   - This means the audio has already been cleaned, resampled (to 16kHz), and divided into smaller chunks for efficient processing.
2. **ASR Model (Whisper Fine-Tuned)**
   - The audio is passed into the **ASR Model block**, which is based on a **fine-tuned Whisper model**.
   - This model is specifically trained to understand **Gujarati speech patterns**.
3. **Transcription Process**

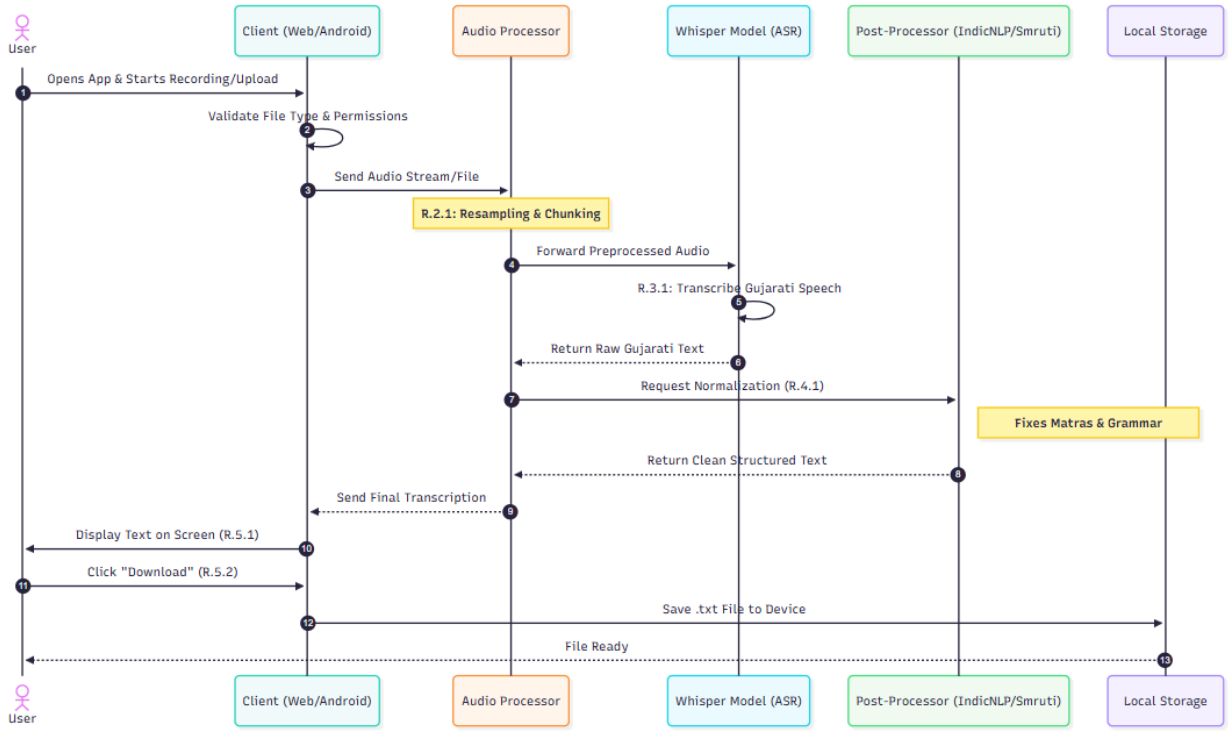
Inside the model, the function:

```
transcribe(audio chunk): String
```

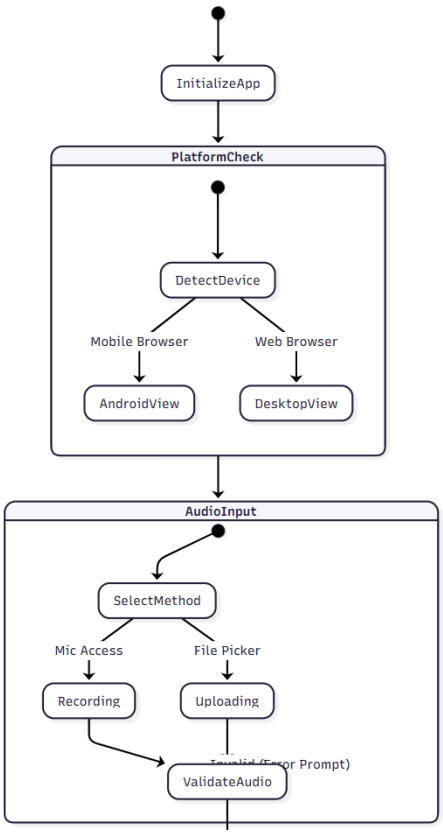
   - converts each audio chunk into **text format**.

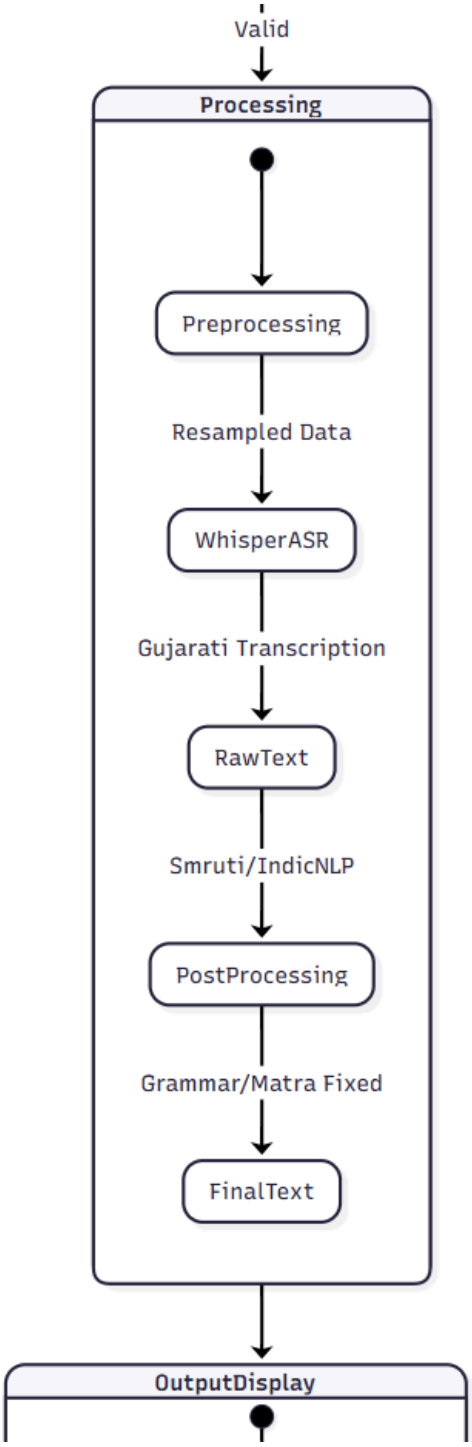
4. **Output: Raw Gujarati Text**
   - The model generates **raw Gujarati text** from the audio.
   - At this stage, the text may still contain minor errors or formatting issues.
5. **Return Output (Display/File)**
   - Finally, the system returns the output:
     - Either displayed on screen
     - Or provided as a downloadable file

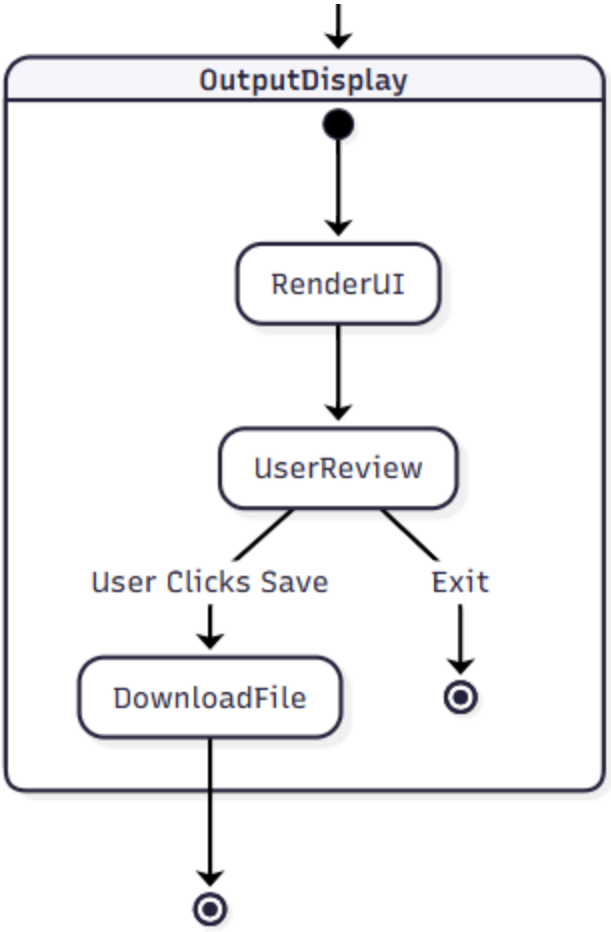
3.3 Sequence Diagram



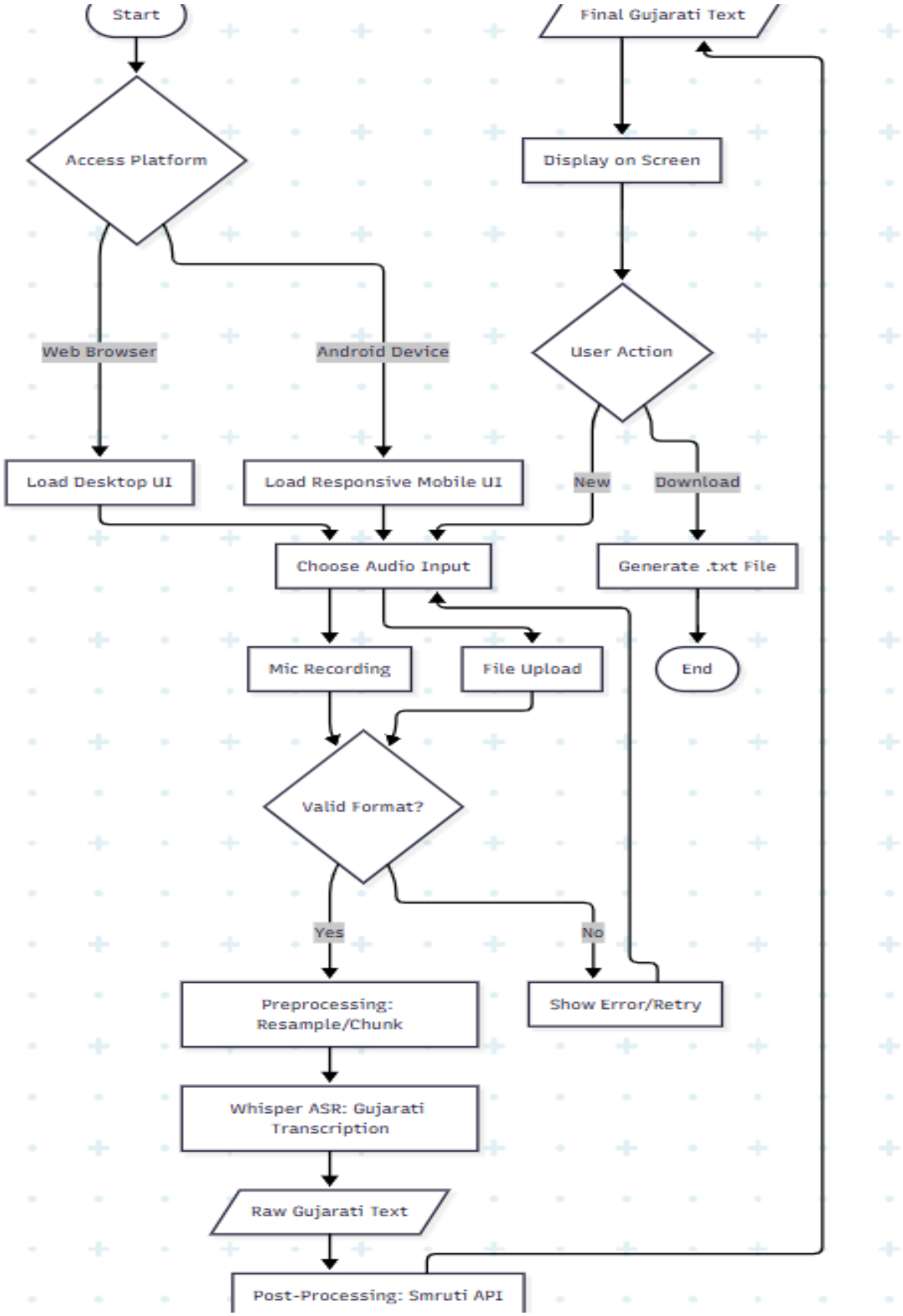
3.4 Activity Diagram




Valid
Processing
Preprocessing
Resampled Data
WhisperASR
Gujarati Transcription
RawText
Smruti/IndicNLP
PostProcessing
Grammar/Matra Fixed
FinalText
OutputDisplay


OutputDisplay
RenderUI
UserReview
User Clicks Save
Exit
DownloadFile

3.5 Flow chart



4. Implementation

4.1 Overview

The Gujarati Custom ASR Pipeline is implemented using a **two-stage architecture** that combines speech recognition and linguistic processing to achieve accurate transcription of Gujarati audio. The system is developed using Python and integrates machine learning models with a web-based interface for user interaction.

4.2 Implementation Details

1. Audio Input Handling

- Users provide input through **audio upload or live recording** (desktop or Android).
- The system accepts formats like WAV and MP3.
- Audio is preprocessed before passing to the model.

2. Audio Preprocessing

- Audio is **resampled to 16kHz** for consistency.
- Long audio is split into **30-second chunks with overlap**.
- Noise and distortion are minimized for better recognition.

3. ASR Model Integration

- The system uses a **fine-tuned Whisper-Small model**.
- Integrated using the Hugging Face Transformers pipeline.
- The model processes audio chunks and generates **raw Gujarati text**.

4. Text Post-Processing

- Raw output is passed through **IndicNLP / Smruti API**.

- Fixes:
    - Unicode errors
    - Matra placement
    - Grammar inconsistencies
- Produces **clean and structured Gujarati text**.

5. Web Interface Implementation

- Built using Gradio.
- Provides:
    - Audio upload option
    - Microphone recording
    - Real-time transcription display
- Accessible via browser on **desktop and Android devices**.

6. Model Hosting and Loading

- Model is hosted on Hugging Face Hub.
- Automatically downloaded when the system runs for the first time.
- Cached locally for faster future use.

7. Output Generation

- Final transcription is:
    - Displayed on the interface
    - Available for download as a text file

4.3 Tools and Technologies Used

- **Programming Language:** Python
- **ML Framework:** PyTorch
- **Model Library:** Hugging Face Transformers
- **UI Framework:** Gradio

- **Audio Processing:** FFmpeg / Librosa
- **NLP Tools:** IndicNLP

4.4 Deployment

- The system is deployed **locally** using Python runtime.
- Users access it via browser (localhost).
- Compatible with **Windows, Linux, macOS, and Android browsers**.

5 Testing

5.1 Overview

Testing is performed to ensure that the Gujarati Custom ASR Pipeline works correctly, efficiently, and reliably. The system is tested at different levels including functional testing, performance testing, and accuracy evaluation to verify that all components operate as expected.

5.2 Types of Testing

1. Unit Testing

- Individual modules such as audio preprocessing, model inference, and text post-processing are tested separately.
- Ensures each component works correctly in isolation.

2. Integration Testing

- Verifies the interaction between modules:
  - Audio input → preprocessing
  - Preprocessing → ASR model
  - ASR model → text normalization
- Ensures smooth data flow across the pipeline.

3. System Testing

- Entire system is tested as a whole.
- Checks end-to-end functionality:
  - Upload/record audio
  - Transcription
  - Output display

4. User Interface Testing

- Ensures the web interface (Gradio) is:
  - User-friendly
  - Responsive (desktop + Android)
  - Error-free

5. Performance Testing

- Evaluates system speed and efficiency.
- Tests:
  - Processing time for audio files
  - Handling of long audio inputs
- Ensures no memory overflow due to chunking mechanism.

6. Accuracy Testing (Important)

- Measures transcription accuracy using **Word Error Rate (WER)**.

Results:

- Baseline Model: 36% WER
- Fine-Tuned Model: 27% WER
- Final Pipeline: **16% WER** 🎉

5.3 Test Cases

Test Case	Input	Expected Output	Result
Audio Upload	Valid Gujarati audio file	Correct transcription	Passed
Audio Recording	Microphone input	Text generated	Passed
Noisy Audio	Background noise audio	Partial accurate text	Passed
Invalid File	Unsupported format	Error message	Passed

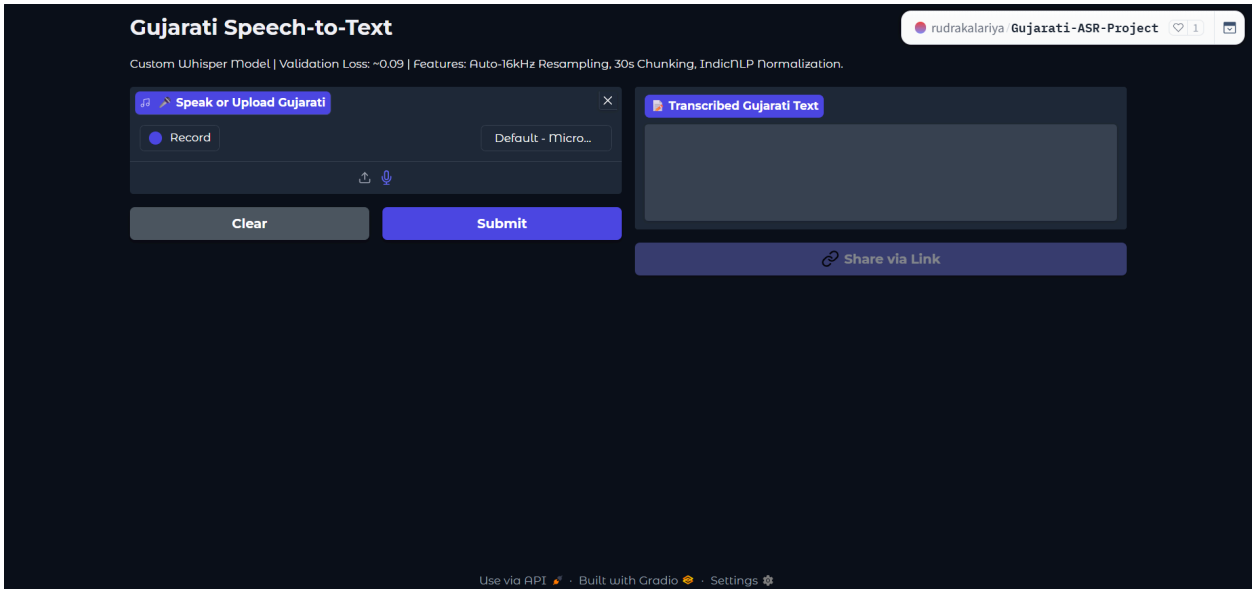
5.4 Tools Used for Testing

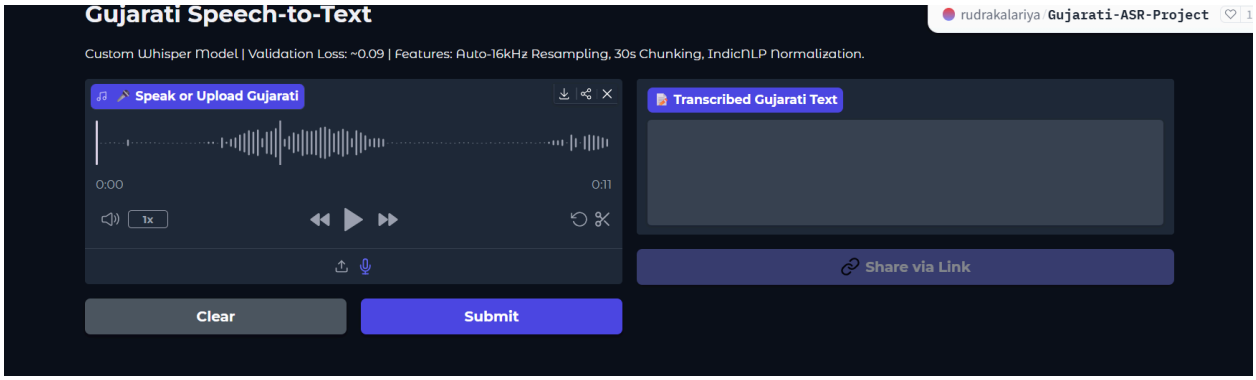
- Python testing scripts
- Manual testing through web interface
- Evaluation datasets (Kathbath, FLEURS)

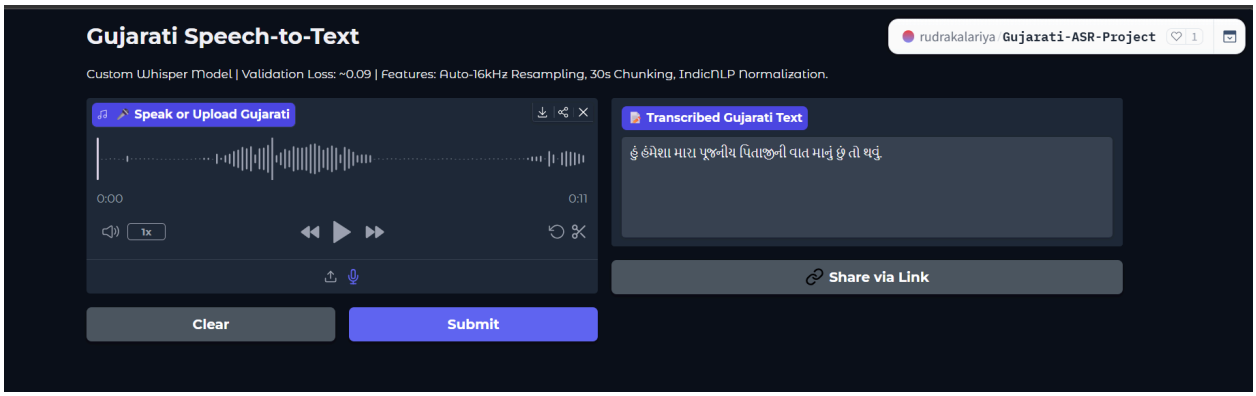
5.5 Conclusion of Testing

The system successfully passes all major test cases and performs efficiently across different environments. The integration of acoustic modeling with linguistic post-processing significantly improves transcription accuracy and reliability.

6. Screenshots






Gujarati Speech-to-Text
rudrakalariya Gujarati-ASR-Project
1
Custom Whisper Model | Validation Loss: ~0.09 | Features: Auto-16kHz Resampling, 30s Chunking, IndicNLP Normalization.
Speak or Upload Gujarati
0:00
0:11
1x
Transcribed Gujarati Text
હું હંમેશા મારા પૂજનીય પિતાજીની વાત માનું છું તો થવું
Share via Link
Clear
Submit

7. Conclusion

The Gujarati Custom ASR Pipeline successfully demonstrates the implementation of an efficient and accurate **speech-to-text system** tailored specifically for the Gujarati language. By combining a **fine-tuned Whisper-based acoustic model** with **linguistic post-processing techniques**, the system effectively addresses key challenges such as phonetic complexity, noisy audio, and Unicode inconsistencies.

The two-stage architecture significantly improves transcription quality, reducing the Word Error Rate (WER) from **36% in the baseline model to 16% in the final pipeline**, which highlights the effectiveness of the proposed approach. The integration of preprocessing techniques like resampling and chunking further enhances system performance and scalability.

The platform provides a **user-friendly web interface** that is accessible on both desktop and Android devices, making it practical for real-world usage. It supports applications such as transcription services, content creation, accessibility tools, and voice-based systems in regional languages.

In conclusion, this project contributes to the advancement of **regional language processing** and promotes the use of Gujarati in digital platforms. Future enhancements may include real-time transcription, support for additional languages, and deployment as a dedicated mobile application.